Caminos mínimos y modelado

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2do Cuatrimestre de 2026

Contenidos

1. DAG
2. Bellman–Ford
3. Dijkstra
4. Floyd–Warshall

Subestructura óptima de los caminos mínimos

Propiedad

Todo subcamino de un camino mínimo también es mínimo entre sus extremos. Es decir, si u pertenece a un camino mínimo de s a v, entonces $d(s, v) = d(s, u) + d(u, v)$. Al considerar todos los posibles vértices intermedios:

$$d(s, v) = \min_{u \in V} \big(d(s, u) + d(u, v)\big).$$

[1] Si se permiten pesos que generan ciclos negativos, hallar un camino simple de costo mínimo entre s y t es NP-difícil; decidir si existe uno de costo a lo sumo B es NP-completo.

Subestructura óptima de los caminos mínimos

Propiedad

Todo subcamino de un camino mínimo también es mínimo entre sus extremos. Es decir, si u pertenece a un camino mínimo de s a v, entonces $d(s, v) = d(s, u) + d(u, v)$. Al considerar todos los posibles vértices intermedios:

$$d(s, v) = \min_{u \in V} \big(d(s, u) + d(u, v)\big).$$

Cómo usa esta propiedad Bellman–Ford

Si $d_k(v)$ es el costo mínimo de un camino de s a v con a lo sumo k aristas, se obtiene la recurrencia

$$d_k(v) = \min \left\{ d_{k-1}(v),\ \min_{(u,v) \in E} \big(d_{k-1}(u) + w(u, v)\big) \right\}.$$

1

[1] Si se permiten pesos que generan ciclos negativos, hallar un camino simple de costo mínimo entre s y t es NP-difícil; decidir si existe uno de costo a lo sumo B es NP-completo.

Caminos en un DAG

Sea $G = (V, E)$ un digrafo acíclico con pesos $w : E \to \mathbb{R}$ y un vértice origen s. Escribimos $n = |V|$ y $m = |E|$.

1. Calcular el peso de un camino **mínimo** desde s hasta cada vértice en tiempo $O(n + m)$.
2. Adaptar el algoritmo para calcular caminos **máximos**.
3. Reconstruir un camino que alcance el peso calculado.

Práctica 8, ejercicio 3. Cormen et al., 4.a edición, sección 22.2.

Caminos en un DAG

Sea $G = (V, E)$ un digrafo acíclico con pesos $w : E \to \mathbb{R}$ y un vértice origen s. Escribimos $n = |V|$ y $m = |E|$.

1. Calcular el peso de un camino **mínimo** desde s hasta cada vértice en tiempo $O(n + m)$.
2. Adaptar el algoritmo para calcular caminos **máximos**.
3. Reconstruir un camino que alcance el peso calculado.

Recordar

En un DAG, un recorrido dirigido no repite vértices. Todo camino tiene a lo sumo $n - 1$ aristas.

Práctica 8, ejercicio 3. Cormen et al., 4.a edición, sección 22.2.

Orden topológico y programación dinámica

Un orden topológico $v_1, \ldots, v_n$ satisface que si $(v_i, v_j) \in E$ entonces $i < j$.

- Cada camino avanza en ese orden.
- Al llegar a v_j, ya procesamos todos sus posibles predecesores.

Orden topológico y programación dinámica

Un orden topológico $v_1, \ldots, v_n$ satisface que si $(v_i, v_j) \in E$ entonces $i < j$.

- Cada camino avanza en ese orden.
- Al llegar a v_j, ya procesamos todos sus posibles predecesores.

Si $\delta(s, v)$ es el peso mínimo desde s hasta v, entonces

$$\delta(s, s) = 0, \text{ y } \delta(s, v) = \min_{(u,v) \in E} \{\delta(s, u) + w(u, v)\} \quad (v \neq s).$$

Usamos $\min \varnothing = +\infty$.

Orden topológico y programación dinámica

Un orden topológico $v_1, \ldots, v_n$ satisface que si $(v_i, v_j) \in E$ entonces $i < j$.

- Cada camino avanza en ese orden.
- Al llegar a v_j, ya procesamos todos sus posibles predecesores.

Si $\delta(s, v)$ es el peso mínimo desde s hasta v, entonces

$$\delta(s, s) = 0, \text{ y } \delta(s, v) = \min_{(u,v) \in E} \{\delta(s, u) + w(u, v)\} \quad (v \neq s).$$

Usamos $\min \varnothing = +\infty$.

Dependencias de los subproblemas

Para calcular el valor de v alcanza con conocer los valores de los vértices que tienen una arista hacia v.

Relajación de una arista

Mantenemos un valor $d[v]$ y un predecesor $\pi[v]$ para cada vértice.

Inicialización

$d[s] = 0, d[v] = +\infty$ $(v \neq s)$ y $\pi[v] = \text{NIL} \forall v$.

Todo valor finito $d[v]$ es el peso de un camino encontrado desde s.

RELAJAR(u, v)

1: **si** $d[u] \neq +\infty$ **y** $d[v] > d[u] + w(u, v)$ **entonces**
2: $\quad d[v] \leftarrow d[u] + w(u, v)$
3: $\quad \pi[v] \leftarrow u$
4: **fin si**

Comparamos el costo de un camino actual con uno que llega a v pasando por u.

Relajación de una arista

Mantenemos un valor $d[v]$ y un predecesor $\pi[v]$ para cada vértice.

Inicialización

$d[s] = 0, d[v] = +\infty$ $(v \neq s)$ y $\pi[v] = \text{NIL} \forall v$.

Todo valor finito $d[v]$ es el peso de un camino encontrado desde s.

RELAJAR(u, v)

1: **si** $d[u] \neq +\infty$ **y** $d[v] > d[u] + w(u, v)$ **entonces**
2: $\quad d[v] \leftarrow d[u] + w(u, v)$
3: $\quad \pi[v] \leftarrow u$
4: **fin si**

Comparamos el costo de un camino actual con uno que llega a v pasando por u.

Efecto

Una relajación disminuye $d[v]$ cuando encuentra un camino mejor. Cada relajación cuesta $O(1)$.

Caminos mínimos en un DAG

1: $L \leftarrow$ ORDEN-TOPOLOGICO(G)
2: $d[v] \leftarrow +\infty$ y $\pi[v] \leftarrow$ NIL para todo $v \in V$
3: $d[s] \leftarrow 0$
4: **para** cada u en el orden L **hacer**
5: **para** cada $v \in \text{Adj}[u]$ **hacer**
6: RELAJAR(u, v)
7: **fin para**
8: **fin para**
9: **devolver** (d, π)

Adaptación de DAG-SHORTEST-PATHS, Cormen et al., sección 22.2.

Caminos mínimos en un DAG

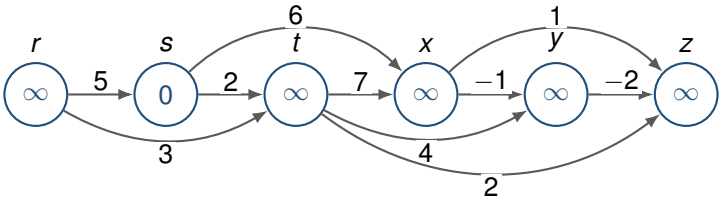
1: $L \leftarrow$ ORDEN-TOPOLOGICO(G)
2: $d[v] \leftarrow +\infty$ y $\pi[v] \leftarrow$ NIL para todo $v \in V$
3: $d[s] \leftarrow 0$
4: **para** cada u en el orden L **hacer**
5: **para** cada $v \in \text{Adj}[u]$ **hacer**
6: RELAJAR(u, v)
7: **fin para**
8: **fin para**
9: **devolver** (d, π)

Un único barrido

Procesamos cada vértice una vez y relajamos cada arista una vez. El origen s puede aparecer en cualquier posición de L.

Adaptación de DAG-SHORTEST-PATHS, Cormen et al., sección 22.2.

Ejemplo: inicialización

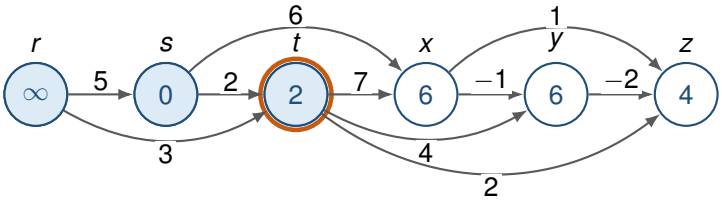


Orden topológico: r, s, t, x, y, z. Origen: s.

- Los números dentro de los vértices representan d.
- Inicialmente, $d[s] = 0$ y los demás valores son $+\infty$.
- Al procesar r, sus aristas no producen cambios: r es inalcanzable desde s.

Grafo de la figura 22.5 de Cormen et al., 4.a edición.

Ejemplo: procesamos s y luego t



Al procesar s

$$d[t] \leftarrow 0 + 2 = 2, \qquad \pi[t] \leftarrow s,$$

$$d[x] \leftarrow 0 + 6 = 6, \qquad \pi[x] \leftarrow s.$$

Al procesar t

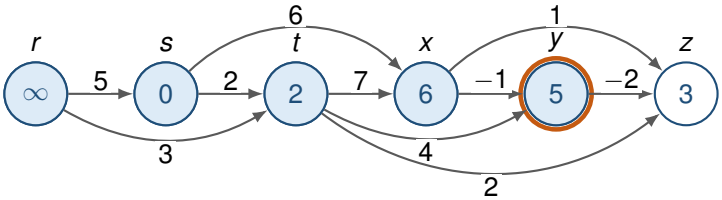
$$d[x] = \text{mín}\{6, 2 + 7\} = 6, d[y] \leftarrow 2 + 4 = 6 \text{ y}$$

$$d[z] \leftarrow 2 + 2 = 4.$$

Ahora $\pi[y] = \pi[z] = t$.

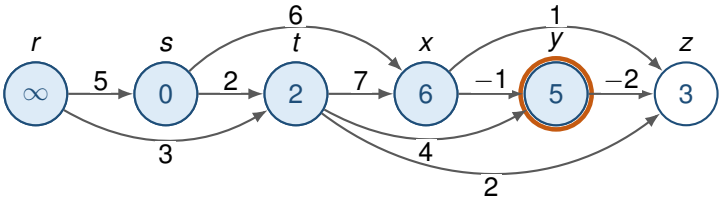
Celeste: vértices procesados. Naranja: vértice de la última iteración.

Ejemplo: las aristas negativas mejoran la solución



- Al procesar x: $d[y] = \text{mín}\{6, 6-1\} = 5$ y $\pi[y] = x$.
- La arista (x, z) no mejora $d[z]$: $\text{mín}\{4, 6+1\} = 4$.
- Al procesar y: $d[z] = \text{mín}\{4, 5-2\} = 3$ y $\pi[z] = y$.
- z no tiene aristas salientes.

Ejemplo: las aristas negativas mejoran la solución



- Al procesar x: $d[y] = \text{mín}\{6, 6-1\} = 5$ y $\pi[y] = x$.
- La arista (x, z) no mejora $d[z]$: $\text{mín}\{4, 6+1\} = 4$.
- Al procesar y: $d[z] = \text{mín}\{4, 5-2\} = 3$ y $\pi[z] = y$.
- z no tiene aristas salientes.

Camino mínimo desde s hasta z

$s \to x \to y \to z$, con peso $6 - 1 - 2 = 3$.

Correctitud: las aristas de un camino se relajan en orden

Sea v alcanzable desde s y sea $P = s = v_0, v_1, \ldots, v_k = v$ un camino mínimo, entonces cada prefijo de P también es mínimo.

Propiedad de relajación de caminos y prueba del teorema 22.5 de Cormen.

Correctitud: las aristas de un camino se relajan en orden

Sea v alcanzable desde s y sea $P = s = v_0, v_1, \ldots, v_k = v$ un camino mínimo, entonces cada prefijo de P también es mínimo.

1. El orden topológico ubica a $v_0, \ldots, v_k$ en ese orden.

Propiedad de relajación de caminos y prueba del teorema 22.5 de Cormen.

Correctitud: las aristas de un camino se relajan en orden

Sea v alcanzable desde s y sea $P = s = v_0, v_1, \ldots, v_k = v$ un camino mínimo, entonces cada prefijo de P también es mínimo.

1. El orden topológico ubica a $v_0, \ldots, v_k$ en ese orden.
2. Al procesar v_i, relajamos (v_i, v_{i+1}).

Propiedad de relajación de caminos y prueba del teorema 22.5 de Cormen.

Correctitud: las aristas de un camino se relajan en orden

Sea v alcanzable desde s y sea $P = s = v_0, v_1, \ldots, v_k = v$ un camino mínimo, entonces cada prefijo de P también es mínimo.

1. El orden topológico ubica a $v_0, \ldots, v_k$ en ese orden.
2. Al procesar v_i, relajamos (v_i, v_{i+1}).
3. Si $d[v_i] = \delta(s, v_i)$, la relajación obtiene $d[v_{i+1}] \leq d[v_i] + w(v_i, v_{i+1}) = \delta(s, v_{i+1})$.

Propiedad de relajación de caminos y prueba del teorema 22.5 de Cormen.

Correctitud: las aristas de un camino se relajan en orden

Sea v alcanzable desde s y sea $P = s = v_0, v_1, \ldots, v_k = v$ un camino mínimo, entonces cada prefijo de P también es mínimo.

1. El orden topológico ubica a $v_0, \ldots, v_k$ en ese orden.
2. Al procesar v_i, relajamos (v_i, v_{i+1}).
3. Si $d[v_i] = \delta(s, v_i)$, la relajación obtiene $d[v_{i+1}] \leq d[v_i] + w(v_i, v_{i+1}) = \delta(s, v_{i+1})$.
4. Todo valor finito es el peso de un camino, por lo que siempre $d[v_{i+1}] \geq \delta(s, v_{i+1})$. Entonces hay igualdad.

Propiedad de relajación de caminos y prueba del teorema 22.5 de Cormen.

Complejidad temporal y espacial

Suponemos listas de adyacencia y acceso al peso de cada arista en $O(1)$.

Operación	Tiempo
Ordenamiento topológico	$O(n+m)$
Inicialización de d y π	$O(n)$
Barrido de los n vértices	$O(n)$
Todas las relajaciones	$O(m)$

El trabajo de los bucles anidados suma

$$\sum_{u \in V} |\operatorname{Adj}[u]| = m.$$

- **Tiempo total:** $O(n+m)$.
- **Espacio auxiliar:** $O(n)$ para d, π, el orden y el recorrido que lo calcula.
- **Espacio total, incluyendo el grafo:** $O(n+m)$.

Caminos máximos: las modificaciones

Sea $D[v]$ el peso máximo desde s hasta v.

	Mínimos	Máximos
Valor inicial del origen	0	0
Valor inicial de los demás	$+\infty$	$-\infty$
Condición de mejora	$d[v] > d[u] + w(u, v)$	$D[v] < D[u] + w(u, v)$
Inalcanzable	$+\infty$	$-\infty$

Usamos el mismo orden topológico y actualizamos también π.

Caminos máximos: las modificaciones

Sea $D[v]$ el peso máximo desde s hasta v.

	Mínimos	Máximos
Valor inicial del origen	0	0
Valor inicial de los demás	$+\infty$	$-\infty$
Condición de mejora	$d[v] > d[u] + w(u, v)$	$D[v] < D[u] + w(u, v)$
Inalcanzable	$+\infty$	$-\infty$

Usamos el mismo orden topológico y actualizamos también π.

Otra forma de obtenerlos

Cambiar cada peso por $w'(u, v) = -w(u, v)$ y buscar mínimos. Para cada vértice alcanzable, $D[v] = -\delta_{w'}(s, v)$.

Aplicación: gestión de proyectos

Un proyecto se divide en etapas $v_1, \ldots, v_n$. Cada etapa v_i tiene duración $t_i \geq 0$.

- Para comenzar v_i deben haber terminado **todas** las etapas de $N(v_i) \subseteq \{v_1, \ldots, v_{i-1}\}$.
- v_1 marca el inicio: $t_1 = 0$ y todas las demás etapas la requieren.
- v_n marca el final: $t_n = 0$ y requiere que terminen todas las demás.

Las etapas cuyas dependencias ya terminaron pueden ejecutarse en paralelo.

Aplicación: gestión de proyectos

Un proyecto se divide en etapas $v_1, \ldots, v_n$. Cada etapa v_i tiene duración $t_i \geq 0$.

- Para comenzar v_i deben haber terminado **todas** las etapas de $N(v_i) \subseteq \{v_1, \ldots, v_{i-1}\}$.
- v_1 marca el inicio: $t_1 = 0$ y todas las demás etapas la requieren.
- v_n marca el final: $t_n = 0$ y requiere que terminen todas las demás.

Las etapas cuyas dependencias ya terminaron pueden ejecutarse en paralelo.

Etapas críticas

Una etapa es **crítica** si cualquier atraso en ella retrasa la finalización del proyecto.

Aplicación: gestión de proyectos

Un proyecto se divide en etapas $v_1, \ldots, v_n$. Cada etapa v_i tiene duración $t_i \geq 0$.

- Para comenzar v_i deben haber terminado **todas** las etapas de $N(v_i) \subseteq \{v_1, \ldots, v_{i-1}\}$.
- v_1 marca el inicio: $t_1 = 0$ y todas las demás etapas la requieren.
- v_n marca el final: $t_n = 0$ y requiere que terminen todas las demás.

Las etapas cuyas dependencias ya terminaron pueden ejecutarse en paralelo.

Etapas críticas

Una etapa es **crítica** si cualquier atraso en ella retrasa la finalización del proyecto.

Objetivo

Identificar todas las etapas críticas en tiempo lineal en el tamaño de la descripción del proyecto.

Medelado del problema como un DAG

Cada etapa v_i es un vértice y tiene una duración $t_i \geq 0$.

Arcos y pesos

Si la etapa i debe terminar antes de que empiece j, agregamos $v_i \to v_j$ con peso $w(v_i, v_j) = t_j$.

Medelado del problema como un DAG

Cada etapa v_i es un vértice y tiene una duración $t_i \geq 0$.

Arcos y pesos

Si la etapa i debe terminar antes de que empiece j, agregamos $v_i \to v_j$ con peso $w(v_i, v_j) = t_j$.

Como $i < j$, el orden dado es topológico. La etapa v_1 es la fuente y v_n el sumidero, con $t_1 = t_n = 0$. Además, v_1 precede a todas las etapas y todas preceden a v_n.

Medelado del problema como un DAG

Cada etapa v_i es un vértice y tiene una duración $t_i \geq 0$.

Arcos y pesos

Si la etapa i debe terminar antes de que empiece j, agregamos $v_i \to v_j$ con peso $w(v_i, v_j) = t_j$.

Como $i < j$, el orden dado es topológico. La etapa v_1 es la fuente y v_n el sumidero, con $t_1 = t_n = 0$. Además, v_1 precede a todas las etapas y todas preceden a v_n.

Observación

La suma de los pesos de un camino $v_1 \rightsquigarrow v_n$ es la duración de la secuencia de etapas que recorre.

Respetar todas las dependencias

Si $d_1[v_j]$ es el instante más temprano en que termina v_j, entonces v_i puede comenzar recién en $\max_{v_j \in N(v_i)} d_1[v_j]$.

Respetar todas las dependencias

Si $d_1[v_j]$ es el instante más temprano en que termina v_j, entonces v_i puede comenzar recién en $\max_{v_j \in N(v_i)} d_1[v_j]$.
Por lo tanto, termina en

$$d_1[v_i] = t_i + \max_{v_j \in N(v_i)} d_1[v_j] = \max_{v_j \in N(v_i)} \big(d_1[v_j] + w(v_j, v_i)\big).$$

Respetar todas las dependencias

Si $d_1[v_j]$ es el instante más temprano en que termina v_j, entonces v_i puede comenzar recién en $\max_{v_j \in N(v_i)} d_1[v_j]$.
Por lo tanto, termina en

$$d_1[v_i] = t_i + \max_{v_j \in N(v_i)} d_1[v_j] = \max_{v_j \in N(v_i)} \big(d_1[v_j] + w(v_j, v_i)\big).$$

Esperar a que terminen todas

El máximo indica cuándo termina la última etapa requerida. Esta es exactamente la recurrencia de caminos máximos en el DAG.

Detección de las etapas críticas

1. En G, calculamos los caminos máximos desde v_1 y los guardamos $d_1[v_i]$ para cada vértice.
2. Invertimos las aristas conservando sus pesos. En G^R, calculamos los caminos máximos desde v_n y guardamos $d_n[v_i]$.
3. Sea $L = d_1[v_n]$. Para cada etapa real v_i, $1 < i < n$: v_i es crítica si y solo si $d_1[v_i] + d_n[v_i] = L$.

Detección de las etapas críticas

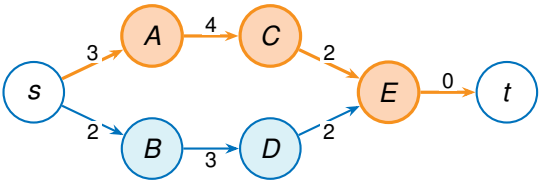
1. En G, calculamos los caminos máximos desde v_1 y los guardamos $d_1[v_i]$ para cada vértice.
2. Invertimos las aristas conservando sus pesos. En G^R, calculamos los caminos máximos desde v_n y guardamos $d_n[v_i]$.
3. Sea $L = d_1[v_n]$. Para cada etapa real v_i, $1 < i < n$: v_i es crítica si y solo si $d_1[v_i] + d_n[v_i] = L$.

Complejidad

Dos recorridos topológicos: $O(n + m)$ tiempo y espacio.

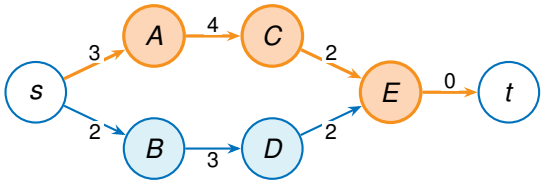
Ejemplo: aplicar el criterio

Los pesos de las aristas son las duraciones de sus extremos finales.



Ejemplo: aplicar el criterio

Los pesos de las aristas son las duraciones de sus extremos finales.



$s = v_1$, $t = v_7$; se omiten arcos redundantes desde s y hacia t.

Distancias y test

Etapa	t_j	d_1	d_n	$d_1 + d_n$
Diseño (A)	3	3	6	9
Compra (B)	2	2	5	7
Montaje (C)	4	7	2	9
Preparación (D)	3	5	2	7
Prueba (E)	2	9	0	9

Resultado

$L = d_1[t] = 9$. Son críticas A, C, E; B, D tienen holgura 2.

Caminos mínimos con pesos negativos

Dado un digrafo ponderado $G = (V, E)$, una función de pesos $w : E \to \mathbb{R}$ y un origen s, queremos calcular el costo mínimo de un camino desde s hasta cada vértice.

- Los pesos pueden ser negativos.
- Si hay un ciclo negativo alcanzable desde s, se puede repetirlo y reducir el costo indefinidamente.
- Bellman–Ford calcula las distancias cuando no hay ciclos negativos alcanzables y también los detecta.

Relajación: mejorar una estimación

Para cada vértice mantenemos una estimación $d[v]$ y un predecesor $\pi[v]$.

Relajar la arista (u, v)

Si $d[u] + w(u, v) < d[v]$, entonces se encontró un camino más barato a v:

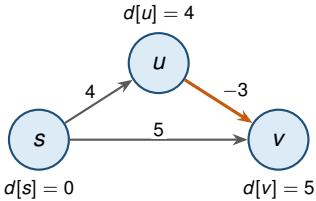
$$d[v] \leftarrow d[u] + w(u, v), \qquad \pi[v] \leftarrow u.$$

- Inicialización: $d[s] = 0$ y $d[v] = +\infty$ para $v \neq s$.
- La condición $d[u] < +\infty$ evita operar sobre vértices todavía inalcanzables.

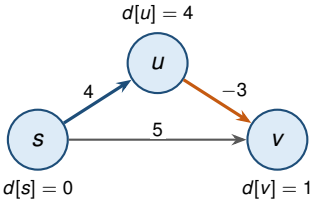
Ilustración: relajar una arista negativa

En el grafo, antes de relajar $u \to v$ ya conocemos $d[u] = 4$ y $d[v] = 5$. El nuevo camino candidato cuesta $d[u] + w(u, v) = 4 - 3 = 1$.

Antes de la relajación



Después de la relajación



Actualización

Como $1 < 5$, asignamos $d[v] \leftarrow 1$ y $\pi[v] \leftarrow u$.

Bellman–Ford

1: **para todo** $v \in V$ **hacer**
2: $\quad d[v] \leftarrow +\infty$; $\pi[v] \leftarrow$ NIL
3: **fin para**
4: $d[s] \leftarrow 0$
5: **para** $i = 1, \ldots, |V| - 1$ **hacer**
6: $\quad$ **para todo** $(u, v) \in E$ **hacer**
7: $\quad\quad$ **si** $d[u] < +\infty$ y $d[u] + w(u, v) < d[v]$ **entonces**
8: $\quad\quad\quad d[v] \leftarrow d[u] + w(u, v)$; $\pi[v] \leftarrow u$
9: $\quad\quad$ **fin si**
10: $\quad$ **fin para**
11: **fin para**
12: **para todo** $(u, v) \in E$ **hacer**
13: $\quad$ **si** $d[u] < +\infty$ y $d[u] + w(u, v) < d[v]$ **entonces**
14: $\quad\quad$ **devolver FALSO** *hay un ciclo negativo alcanzable*
15: $\quad$ **fin si**
16: **fin para**
17: **devolver VERDADERO**

¿Por qué alcanza con $|V| - 1$ pasadas?

Si no hay ciclos negativos alcanzables, existe un camino mínimo simple desde s hasta cada vértice alcanzable.

1. Un camino simple visita cada vértice a lo sumo una vez.
2. Por lo tanto, usa como máximo $|V| - 1$ aristas.
3. Después de i pasadas, Bellman–Ford ya considera caminos de hasta i aristas.
4. Tras $|V| - 1$ pasadas, todas las distancias mínimas están calculadas.
5. Si una arista aún se puede relajar, algún camino sigue mejorando después de ese límite: eso solo puede ocurrir si un ciclo negativo es alcanzable.

Complejidad de Bellman–Ford

Sea $n = |V|$ y $m = |E|$. Suponemos una lista de aristas con acceso $O(1)$ a los extremos y pesos de cada arista.

Operación	Tiempo
Inicialización de d y π	$O(n)$
$n-1$ pasadas sobre todas las aristas	$O((n-1)m)$
Chequeo final de todas las aristas	$O(m)$

Cada pasada recorre una vez las m aristas:

$$(n-1) \cdot O(m) = O((n-1)m).$$

- **Tiempo total:** $O(n + (n-1)m + m) = O(nm)$.
- **Espacio auxiliar:** $O(n)$ para d y π.
- **Espacio total, incluyendo el grafo:** $O(n+m)$.

Ejemplo de ejecución

Usamos este orden de aristas: $s \to t$, $s \to y$, $t \to z$, $x \to t$, $y \to x$, $z \to x$.

Arista	Peso
$s \to t$	6
$s \to y$	7
$t \to z$	-4
$x \to t$	-2
$y \to x$	-3
$z \to x$	7

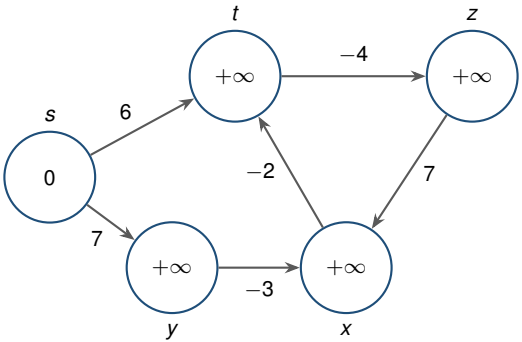
Pasada	$d[s]$	$d[t]$	$d[x]$	$d[y]$	$d[z]$
Inicial	0	∞	∞	∞	∞
1	0	6	4	7	2
2	0	2	4	7	2
3	0	2	4	7	-2
4	0	2	4	7	-2

Las distancias finales son $d[t] = 2$, $d[x] = 4$, $d[y] = 7$ y $d[z] = -2$. En la cuarta pasada no hay cambios.

Ejemplo sin ciclos negativos

Agregamos $z \to x$ con peso 7. El ciclo $x \to t \to z \to x$ cuesta

$$-2 - 4 + 7 = 1.$$



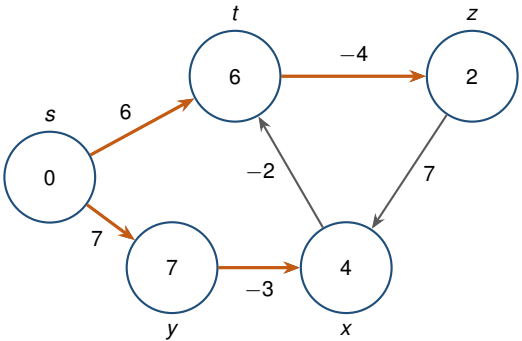
Inicialización: $d[s] = 0$ y las demás distancias valen $+\infty$.

Orden de barrido: $s \to t$, $s \to y$, $t \to z$, $x \to t$, $y \to x$, $z \to x$.

Ejemplo sin ciclos negativos

Agregamos $z \to x$ con peso 7. El ciclo $x \to t \to z \to x$ cuesta

$$-2 - 4 + 7 = 1.$$



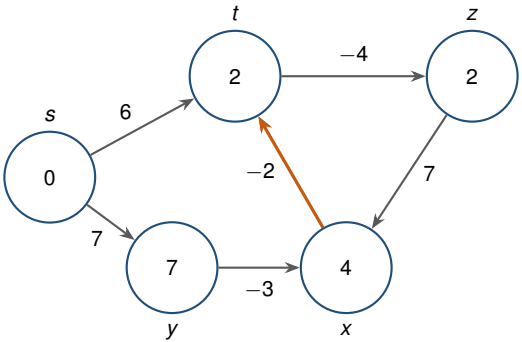
Pasada 1: $d[t] = 6$, $d[y] = 7$, $d[x] = 4$ y $d[z] = 2$.

Orden de barrido: $s \to t$, $s \to y$, $t \to z$, $x \to t$, $y \to x$, $z \to x$.

Ejemplo sin ciclos negativos

Agregamos $z \to x$ con peso 7. El ciclo $x \to t \to z \to x$ cuesta

$$-2 - 4 + 7 = 1.$$



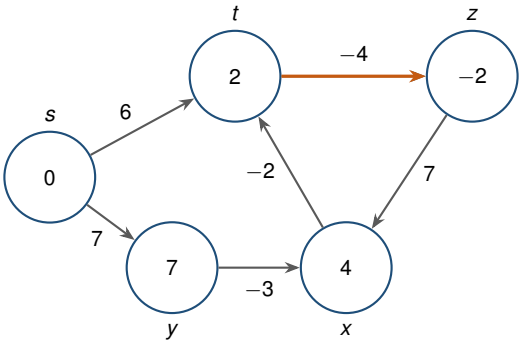
Pasada 2: $x \to t$ mejora $d[t]$ de 6 a $4 - 2 = 2$.

Orden de barrido: $s \to t$, $s \to y$, $t \to z$, $x \to t$, $y \to x$, $z \to x$.

Ejemplo sin ciclos negativos

Agregamos $z \to x$ con peso 7. El ciclo $x \to t \to z \to x$ cuesta

$$-2 - 4 + 7 = 1.$$



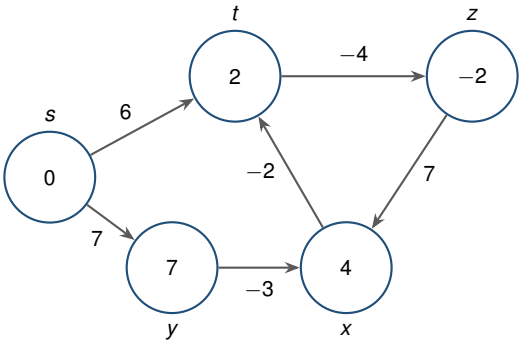
Pasada 3: $t \to z$ mejora $d[z]$ de 2 a $2 - 4 = -2$.

Orden de barrido: $s \to t$, $s \to y$, $t \to z$, $x \to t$, $y \to x$, $z \to x$.

Ejemplo sin ciclos negativos

Agregamos $z \to x$ con peso 7. El ciclo $x \to t \to z \to x$ cuesta

$$-2 - 4 + 7 = 1.$$

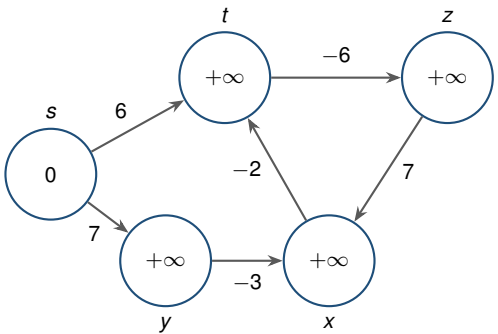


Pasada 4 sin cambios; $z \to x$ propone $-2 + 7 = 5 > 4$. No hay ciclo negativo.

Orden de barrido: $s \to t,\ s \to y,\ t \to z,\ x \to t,\ y \to x,\ z \to x.$

Mismo grafo: Bellman–Ford detecta un ciclo negativo

Cambiamos $w(t, z)$ a -6; el ciclo $x \to t \to z \to x$ pesa $-2 - 6 + 7 = -1$.



Orden de barrido: $s \to t$, $s \to y$, $y \to x$, $x \to t$, $t \to z$, $z \to x$.

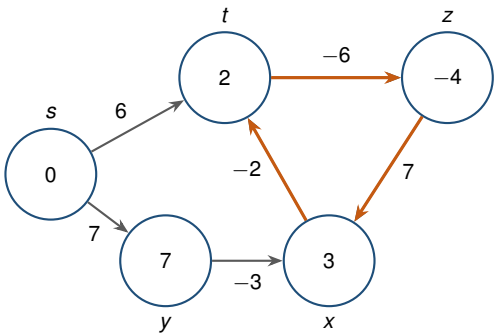
Inicialización

Arrancamos con $d[s] = 0$, $d[v] = +\infty$ para $v \neq s$ y $\pi[v] = \text{NIL}$ para todo v.

Cada vuelta del ciclo reduce el costo en 1.

Mismo grafo: Bellman–Ford detecta un ciclo negativo

Cambiamos $w(t, z)$ a -6; el ciclo $x \to t \to z \to x$ pesa $-2 - 6 + 7 = -1$.



Orden de barrido: $s \to t$, $s \to y$, $y \to x$, $x \to t$, $t \to z$, $z \to x$.

Pasada 1

$d[t] = 2$, $d[y] = 7$, $d[x] = 3$, $d[z] = -4$.

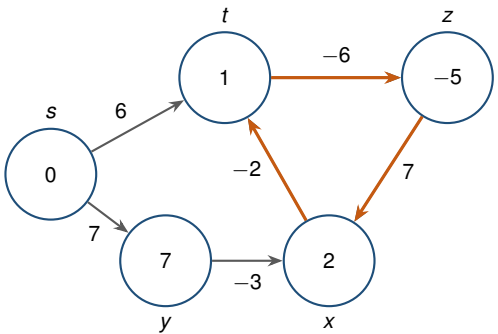
Predecesores

$\pi[t] = x$, $\pi[y] = s$, $\pi[x] = z$, $\pi[z] = t$.

Cada vuelta del ciclo reduce el costo en 1.

Mismo grafo: Bellman–Ford detecta un ciclo negativo

Cambiamos $w(t, z)$ a -6; el ciclo $x \to t \to z \to x$ pesa $-2 - 6 + 7 = -1$.



Orden de barrido: $s \to t$, $s \to y$, $y \to x$, $x \to t$, $t \to z$, $z \to x$.

Pasada 2

$d[t] = 1$, $d[y] = 7$, $d[x] = 2$, $d[z] = -5$.

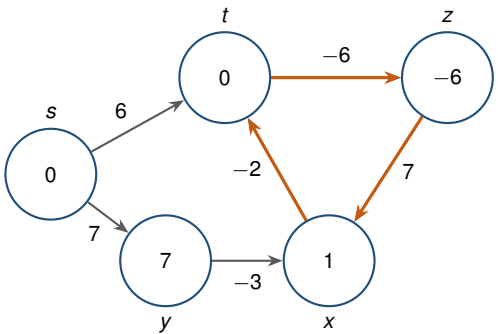
Predecesores

$\pi[t] = x$, $\pi[y] = s$, $\pi[x] = z$, $\pi[z] = t$.

Cada vuelta del ciclo reduce el costo en 1.

Mismo grafo: Bellman–Ford detecta un ciclo negativo

Cambiamos $w(t, z)$ a -6; el ciclo $x \to t \to z \to x$ pesa $-2 - 6 + 7 = -1$.



Orden de barrido: $s \to t$, $s \to y$, $y \to x$, $x \to t$, $t \to z$, $z \to x$.

Pasada 3

$d[t] = 0$, $d[y] = 7$, $d[x] = 1$, $d[z] = -6$.

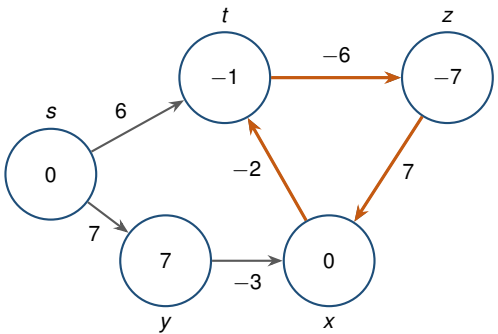
Predecesores

$\pi[t] = x$, $\pi[y] = s$, $\pi[x] = z$, $\pi[z] = t$.

Cada vuelta del ciclo reduce el costo en 1.

Mismo grafo: Bellman–Ford detecta un ciclo negativo

Cambiamos $w(t, z)$ a -6; el ciclo $x \to t \to z \to x$ pesa $-2 - 6 + 7 = -1$.



Orden de barrido: $s \to t$, $s \to y$, $y \to x$, $x \to t$, $t \to z$, $z \to x$.

Pasada 4

$d[t] = -1$, $d[y] = 7$, $d[x] = 0$, $d[z] = -7$.

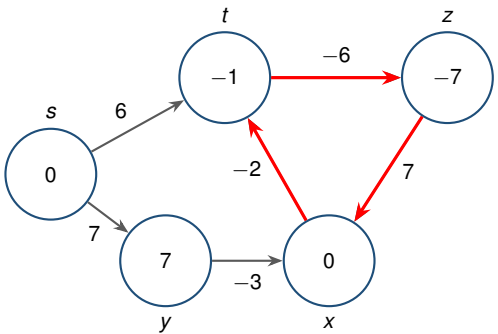
Predecesores

$\pi[t] = x$, $\pi[y] = s$, $\pi[x] = z$, $\pi[z] = t$.

Cada vuelta del ciclo reduce el costo en 1.

Mismo grafo: Bellman–Ford detecta un ciclo negativo

Cambiamos $w(t, z)$ a -6; el ciclo $x \to t \to z \to x$ pesa $-2 - 6 + 7 = -1$.



Orden de barrido: $s \to t$, $s \to y$, $y \to x$, $x \to t$, $t \to z$, $z \to x$.

Chequeo final

$d[x] - 2 = 0 - 2 = -2 < -1 = d[t]$: se puede relajar. Ciclo negativo detectado.

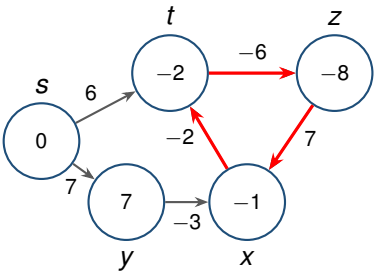
Predecesores

$\pi[t] = x$, $\pi[y] = s$, $\pi[x] = z$, $\pi[z] = t$.

Cada vuelta del ciclo reduce el costo en 1.

Reconstrucción de un ciclo negativo

Sea x el último vértice actualizado en la pasada n. Desde x, seguimos n predecesores; luego seguimos hasta regresar al mismo vértice e invertimos la lista.



Distancias al terminar la pasada 5.

Pasada 5 ($n = 5$)
Tras cuatro pasadas: $d[t] = -1$, $d[z] = -7$, $d[x] = 0$. En la pasada 5 se relajan $x \to t$, $t \to z$ y $z \to x$. El último actualizado es x, con $\pi[t] = x$, $\pi[z] = t$ y $\pi[x] = z$.

Entrar al ciclo
Desde x, seguimos cinco predecesores: x, z, t, x, z, t. Llegamos a t.

Reconstruirlo
La cadena de predecesores es t, x, z, t; al invertirla obtenemos $t \to z \to x \to t$. Su peso es $-6 + 7 - 2 = -1$.

Correctitud de Bellman–Ford

Teorema

Bellman–Ford devuelve **VERDADERO** si y solo si no hay un ciclo negativo alcanzable desde s. Cuando devuelve VERDADERO, $d[v] = \delta(s, v)$ para todo vértice alcanzable v.

Correctitud de Bellman–Ford

Teorema

Bellman–Ford devuelve **VERDADERO** si y solo si no hay un ciclo negativo alcanzable desde s. Cuando devuelve VERDADERO, $d[v] = \delta(s, v)$ para todo vértice alcanzable v.

- Un vértice inalcanzable conserva $d[v] = +\infty$.
- Si existe un ciclo negativo alcanzable, el costo de los recorridos hacia sus vértices no tiene mínimo finito.

Lema: qué garantizan las pasadas

Lema

1. Si $d[v] < +\infty$, su valor es el costo de algún recorrido de s a v. La inicialización y cada relajación conservan esta propiedad.
2. Después de i pasadas, para todo camino P de s a v con a lo sumo i aristas, $d[v] \leq w(P)$.

Lema: qué garantizan las pasadas

Lema

1. Si $d[v] < +\infty$, su valor es el costo de algún recorrido de s a v. La inicialización y cada relajación conservan esta propiedad.
2. Después de i pasadas, para todo camino P de s a v con a lo sumo i aristas, $d[v] \leq w(P)$.

Por qué vale el segundo punto

Se prueba por inducción en la cantidad de aristas de P. Si la última es (u, v), el prefijo hasta u ya quedó cubierto por las pasadas anteriores. Al relajar (u, v), se obtiene $d[v] \leq d[u] + w(u, v) \leq w(P)$.

Sin ciclos negativos: se obtienen las distancias mínimas

Supongamos que no hay ciclos negativos alcanzables desde s.

1. Todo vértice alcanzable tiene un camino mínimo simple: si un recorrido repite un ciclo, ese ciclo tiene peso no negativo y se puede quitar.
2. Un camino simple usa a lo sumo $n - 1$ aristas. Por el lema, al terminar las pasadas, $d[v] \leq \delta(s, v)$.
3. Como cada $d[v]$ finito es el costo de un recorrido, también $\delta(s, v) \leq d[v]$.

Sin ciclos negativos: se obtienen las distancias mínimas

Supongamos que no hay ciclos negativos alcanzables desde s.

1. Todo vértice alcanzable tiene un camino mínimo simple: si un recorrido repite un ciclo, ese ciclo tiene peso no negativo y se puede quitar.
2. Un camino simple usa a lo sumo $n-1$ aristas. Por el lema, al terminar las pasadas, $d[v] \leq \delta(s, v)$.
3. Como cada $d[v]$ finito es el costo de un recorrido, también $\delta(s, v) \leq d[v]$.

Conclusión

Para cada vértice alcanzable, $d[v] = \delta(s, v)$. Además, para cada arista (u, v) con u alcanzable,

$$d[v] = \delta(s, v) \leq \delta(s, u) + w(u, v) = d[u] + w(u, v).$$

El chequeo final no encuentra ninguna arista relajable.

Con ciclo negativo: el chequeo lo detecta

Supongamos que un ciclo negativo C es alcanzable desde s.

- Cada vértice de C se alcanza por un camino simple de a lo sumo $n - 1$ aristas; por eso, al terminar las pasadas, sus distancias son finitas.

Con ciclo negativo: el chequeo lo detecta

Supongamos que un ciclo negativo C es alcanzable desde s.

- Cada vértice de C se alcanza por un camino simple de a lo sumo $n-1$ aristas; por eso, al terminar las pasadas, sus distancias son finitas.
- Si ninguna arista de C pudiera relajarse, para cada $(u, v) \in C$ tendríamos $d[v] \leq d[u] + w(u, v)$.

Con ciclo negativo: el chequeo lo detecta

Supongamos que un ciclo negativo C es alcanzable desde s.

- Cada vértice de C se alcanza por un camino simple de a lo sumo $n-1$ aristas; por eso, al terminar las pasadas, sus distancias son finitas.
- Si ninguna arista de C pudiera relajarse, para cada $(u,v) \in C$ tendríamos $d[v] \leq d[u] + w(u,v)$.
- Al sumar alrededor de C, los términos con $d[\cdot]$ se cancelan y queda $0 \leq \sum_{(u,v)\in C} w(u,v) = w(C) < 0$, contradicción. Por lo tanto, el chequeo encuentra una arista relajable.

Con ciclo negativo: el chequeo lo detecta

Supongamos que un ciclo negativo C es alcanzable desde s.

- Cada vértice de C se alcanza por un camino simple de a lo sumo $n-1$ aristas; por eso, al terminar las pasadas, sus distancias son finitas.
- Si ninguna arista de C pudiera relajarse, para cada $(u,v) \in C$ tendríamos $d[v] \leq d[u] + w(u,v)$.
- Al sumar alrededor de C, los términos con $d[\cdot]$ se cancelan y queda $0 \leq \sum_{(u,v)\in C} w(u,v) = w(C) < 0$, contradicción. Por lo tanto, el chequeo encuentra una arista relajable.

Con ciclo negativo: el chequeo lo detecta

Supongamos que un ciclo negativo C es alcanzable desde s.

- Cada vértice de C se alcanza por un camino simple de a lo sumo $n-1$ aristas; por eso, al terminar las pasadas, sus distancias son finitas.
- Si ninguna arista de C pudiera relajarse, para cada $(u,v) \in C$ tendríamos $d[v] \leq d[u] + w(u,v)$.
- Al sumar alrededor de C, los términos con $d[\cdot]$ se cancelan y queda $0 \leq \sum_{(u,v)\in C} w(u,v) = w(C) < 0$, contradicción. Por lo tanto, el chequeo encuentra una arista relajable.

Conclusión

El chequeo falla exactamente cuando hay un ciclo negativo alcanzable.

Pregunta: ganancia al repetir un circuito

Problema

1. Un conductor se mueve por una red de cabinas. Cada tramo cuesta dinero; al llegar a una cabina, puede pagar un peaje o recibir una suma. Identificar si existe un circuito que, al repetirse, produzca una ganancia neta cada vez mayor al conductor.
2. Desde el departamento de marketing sugieren transformar la idea de cabinas inversas en cabinas mixtas. Cuando un vehículo pasa por una cabina mixta, se le paga a le conductore si se le cobró en la cabina anterior; caso contrario, le conductore paga. Desde marketing sugieren que se le pague cuando la cabina mixta sea la primera cabina recorrida, para bajar los malos humores.

Modelo del sistema original

Asignamos a cada transición el costo del viaje más el costo de la cabina de llegada:

$$w(i,j) = c_{ij} + c_j.$$

Para un ciclo $C = (v_1, v_2, \ldots, v_k, v_1)$:

$$w(C) = \sum_{r=1}^{k} c_{v_r, v_{r+1}} + \sum_{r=1}^{k} c_{v_{r+1}}, \qquad v_{k+1} = v_1.$$

Modelo del sistema original

Asignamos a cada transición el costo del viaje más el costo de la cabina de llegada:

$$w(i,j) = c_{ij} + c_j.$$

Para un ciclo $C = (v_1, v_2, \ldots, v_k, v_1)$:

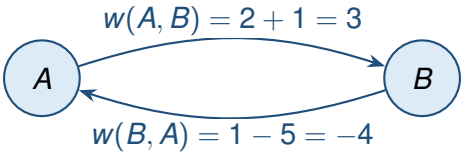
$$w(C) = \sum_{r=1}^{k} c_{v_r, v_{r+1}} + \sum_{r=1}^{k} c_{v_{r+1}}, \qquad v_{k+1} = v_1.$$

Criterio

Hay ganancia ilimitada exactamente cuando existe un ciclo de peso negativo. Repetirlo reduce el costo neto del conductor en cada vuelta.

Un circuito rentable

Supongamos dos cabinas: A paga 5 unidades ($c_A = -5$) y B cobra 1 ($c_B = 1$). Los viajes cuestan $c_{AB} = 2$ y $c_{BA} = 1$.



$$w(A \to B \to A) = 3 + (-4) = -1.$$

El circuito deja una ganancia neta de 1 unidad por vuelta; Bellman–Ford lo detecta.

Resolución: sistema original

1. Construir G con un vértice por cabina y pesos $w(i, j) = c_{ij} + c_j$.
2. Agregar un vértices auxiliar s con aristas de peso 0 a todas las cabinas.
3. Ejecutar Bellman–Ford desde s.
4. Si alguna arista todavía se relaja tras $|V| - 1$ pasadas, existe un circuito rentable; si no, no hay ganancia ilimitada.

Resolución: sistema original

1. Construir G con un vértice por cabina y pesos $w(i, j) = c_{ij} + c_j$.
2. Agregar un vértices auxiliar s con aristas de peso 0 a todas las cabinas.
3. Ejecutar Bellman–Ford desde s.
4. Si alguna arista todavía se relaja tras $|V| - 1$ pasadas, existe un circuito rentable; si no, no hay ganancia ilimitada.

Si hay n cabinas y m transiciones, el vértice auxiliar agrega n aristas y el costo es $O((n+1)(m+n))$. Si hay viaje directo entre cada par de cabinas, $m = \Theta(n^2)$ y el costo es $O(n^3)$.

Extensión: cabinas mixtas

En una cabina mixta j, el cobro depende de lo ocurrido en la cabina anterior. Como $c_j > 0$ es el monto que se cobra:

Estado al llegar a j	Costo en j	Estado al salir de j
La cabina anterior cobró ($b = 1$)	$-c_j$	Pagó ($b' = 0$)
La cabina anterior pagó ($b = 0$)	c_j	Cobró ($b' = 1$)

Extensión: cabinas mixtas

En una cabina mixta j, el cobro depende de lo ocurrido en la cabina anterior. Como $c_j > 0$ es el monto que se cobra:

Estado al llegar a j	Costo en j	Estado al salir de j
La cabina anterior cobró ($b = 1$)	$-c_j$	Pagó ($b' = 0$)
La cabina anterior pagó ($b = 0$)	c_j	Cobró ($b' = 1$)

Recordamos un bit: $b = 1$ si la cabina recién visitada cobró y $b = 0$ si pagó.

Extensión: cabinas mixtas

En una cabina mixta j, el cobro depende de lo ocurrido en la cabina anterior. Como $c_j > 0$ es el monto que se cobra:

Estado al llegar a j	Costo en j	Estado al salir de j
La cabina anterior cobró ($b = 1$)	$-c_j$	Pagó ($b' = 0$)
La cabina anterior pagó ($b = 0$)	c_j	Cobró ($b' = 1$)

Recordamos un bit: $b = 1$ si la cabina recién visitada cobró y $b = 0$ si pagó.

Primera cabina

Si la primera cabina es mixta, se paga al conductor: costo $-c_i$ y estado inicial $b = 0$.

Grafo de estados para las cabinas mixtas

- Estados: (i, b), donde i es la cabina actual y b indica si cobró en esa cabina.
- Desde (i, b), para cada viaje $i \to j$, se agrega una transición al estado que resulte de aplicar la regla de j.
- El peso es c_{ij} más el costo de la cabina j según el estado b.
- En una cabina no mixta, el costo es el original c_j y el estado nuevo indica si cobró o pagó.

El vértice auxiliar se conecta con los estados iniciales válidos. Se usan los costos reales de primera cabina; en particular, una primera cabina mixta paga $-c_i$.

Por qué funciona el estado

Un ciclo en este grafo vuelve a la misma cabina y al mismo bit b. Al repetirlo, las reglas mixtas producen la misma secuencia de cobros y pagos.

Resolución: cabinas mixtas

Ejecutamos Bellman–Ford en el grafo de estados desde el vértice auxiliar.

- Ciclo negativo alcanzable en el grafo de estados implican que se puede repetir una secuencia legal de visitas y ganar sin límite.
- Si no hay ciclos negativos alcanzables entonces no existe un circuito rentable repetible.
- Hay a lo sumo $2n + 1$ vértices y $2m + n$ aristas, contando el vértice auxiliar; el tiempo es $O(n(m + n))$.

Resolución: cabinas mixtas

Ejecutamos Bellman–Ford en el grafo de estados desde el vértice auxiliar.

- Ciclo negativo alcanzable en el grafo de estados implican que se puede repetir una secuencia legal de visitas y ganar sin límite.
- Si no hay ciclos negativos alcanzables entonces no existe un circuito rentable repetible.
- Hay a lo sumo $2n + 1$ vértices y $2m + n$ aristas, contando el vértice auxiliar; el tiempo es $O(n(m + n))$.

Idea del modelo

La variante mixta se resuelve ampliando el estado para representar la dependencia de la cabina anterior.

s

Dijkstra: caminos mínimos con pesos no negativos

El objetivo es calcular $\delta(s, v)$ desde un origen s hasta cada vértice.

Idea greedy

Mantenemos los vértices ya fijados, cuyas distancias consideramos definitivas. En cada paso fijamos el vértice pendiente con menor estimación $d[v]$ y relajamos sus aristas hacia los vértices que aún no fueron fijados.

Dijkstra: caminos mínimos con pesos no negativos

El objetivo es calcular $\delta(s, v)$ desde un origen s hasta cada vértice.

Idea greedy

Mantenemos los vértices ya fijados, cuyas distancias consideramos definitivas. En cada paso fijamos el vértice pendiente con menor estimación $d[v]$ y relajamos sus aristas hacia los vértices que aún no fueron fijados.

Hipótesis esencial

Todos los pesos deben ser no negativos: $w(e) \geq 0$ para cada $e \in E$. Con pesos negativos, una distancia ya fijada podría mejorar más adelante.

Dijkstra: caminos mínimos con pesos no negativos

El objetivo es calcular $\delta(s, v)$ desde un origen s hasta cada vértice.

Idea greedy

Mantenemos los vértices ya fijados, cuyas distancias consideramos definitivas. En cada paso fijamos el vértice pendiente con menor estimación $d[v]$ y relajamos sus aristas hacia los vértices que aún no fueron fijados.

Hipótesis esencial

Todos los pesos deben ser no negativos: $w(e) \geq 0$ para cada $e \in E$. Con pesos negativos, una distancia ya fijada podría mejorar más adelante.

Diferencias con Bellman – Ford

La elección greedy de elegir la menor etiqueta es lo que distingue a Dijkstra de Bellman–Ford.

Algoritmo de Dijkstra

```
1: para v ∈ V hacer
2:     d[v] ← +∞; π[v] ← NIL
3: fin para
4: d[s] ← 0
5: Q ← V                                   ▷ cola de prioridad por d[v]
6: mientras Q ≠ ∅ hacer
7:     u ← ExtractMin(Q)
8:     si d[u] < +∞ entonces
9:         para aristas (u, v) ∈ E con v ∈ Q hacer
10:            si d[u] + w(u, v) < d[v] entonces
11:                d[v] ← d[u] + w(u, v); π[v] ← u
12:            fin si
13:        fin para
14:    fin si
15: fin mientras
```

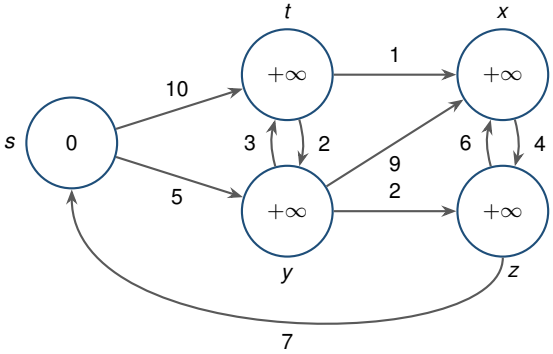
Cola de prioridad

Q contiene los vértices no fijados y los ordena por $d[v]$. EXTRACTMIN devuelve el de menor etiqueta.

- Si $d[u] = +\infty$, u es inalcanzable; no se relajan sus aristas.
- $\pi[v]$ guarda el predecesor de v cuando se mejora $d[v]$.

Ejemplo de ejecución de Dijkstra

El naranja indica el vértice fijado en esta iteración.

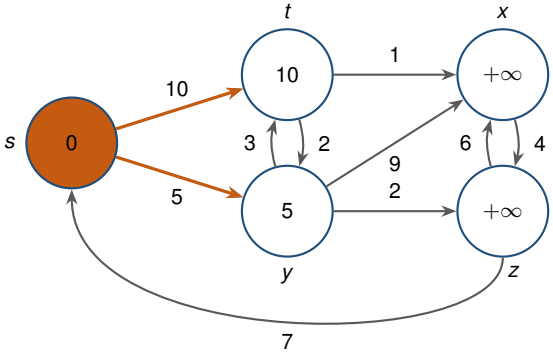


Inicialización: $d[s] = 0$ y las demás etiquetas valen $+\infty$.

Orden de fijación: s, y, z, t, x.

Ejemplo de ejecución de Dijkstra

El naranja indica el vértice fijado en esta iteración.

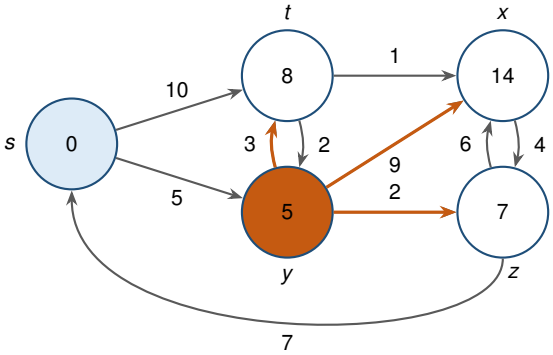


Fijamos s: $d[t] = 10$ y $d[y] = 5$.

Orden de fijación: s, y, z, t, x.

Ejemplo de ejecución de Dijkstra

El naranja indica el vértice fijado en esta iteración.

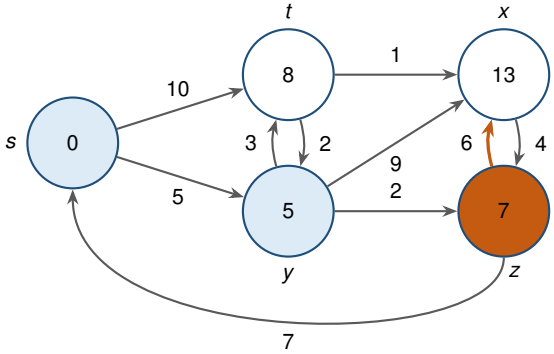


Fijamos y (mínimo 5): $d[t] = 8$, $d[z] = 7$ y $d[x] = 14$.

Orden de fijación: s, y, z, t, x.

Ejemplo de ejecución de Dijkstra

El naranja indica el vértice fijado en esta iteración.

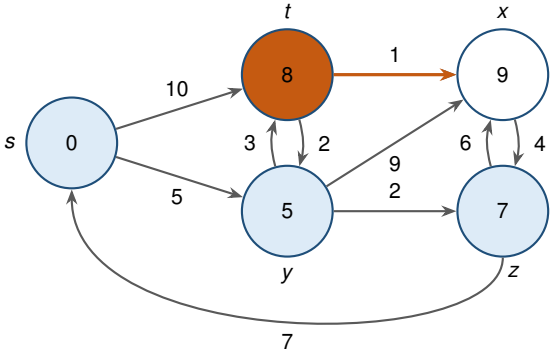


Fijamos z (mínimo 7): $d[x]$ mejora de 14 a 13.

Orden de fijación: $s,\ y,\ z,\ t,\ x$.

Ejemplo de ejecución de Dijkstra

El naranja indica el vértice fijado en esta iteración.

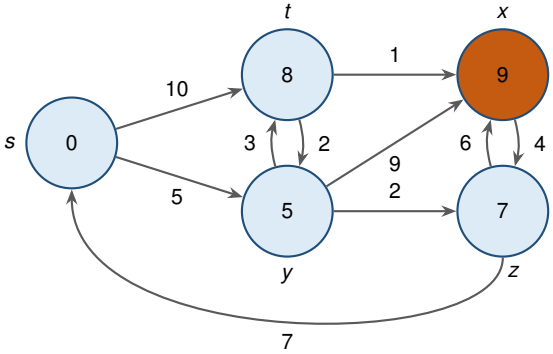


Fijamos t (mínimo 8): $d[x]$ mejora de 13 a 9.

Orden de fijación: s, y, z, t, x.

Ejemplo de ejecución de Dijkstra

El naranja indica el vértice fijado en esta iteración.



Fijamos x (mínimo 9). Distancias finales: $d[t] = 8$, $d[x] = 9$, $d[y] = 5$, $d[z] = 7$.

Orden de fijación: $s,\ y,\ z,\ t,\ x$.

Correctitud de Dijkstra: invariante

Sea S el conjunto de vértices que ya fueron extraídos de Q y fijados.

Lema (invariante)

Cada vértice fijado tiene su distancia correcta; es decir, $d[u] = \delta(s, u)$ para todo $u \in S$. Además, toda etiqueta finita $d[v]$ es el costo de algún recorrido desde s hasta v. Por eso, siempre $\delta(s, v) \leq d[v]$.

- **Inicio:** antes de extraer, $S = \emptyset$. La primera extracción es s, pues $d[s] = 0$ y el resto vale $+\infty$; así $d[s] = \delta(s, s) = 0$.
- **Paso:** falta probar que el vértice u de menor etiqueta en Q ya tiene distancia correcta cuando se fija.

Correctitud de Dijkstra: por qué se fija el mínimo

Sea u el vértice de menor etiqueta en Q y sea P un camino mínimo de s a u. Como los pesos son no negativos, elegimos P simple.

1. Sea y el primer vértice de P que no está en S, y sea x su predecesor. Entonces $x \in S$.

Correctitud de Dijkstra: por qué se fija el mínimo

Sea u el vértice de menor etiqueta en Q y sea P un camino mínimo de s a u. Como los pesos son no negativos, elegimos P simple.

1. Sea y el primer vértice de P que no está en S, y sea x su predecesor. Entonces $x \in S$.
2. Al fijar x, se relajó (x, y). Por el invariante, $d[y] \leq \delta(s, x) + w(x, y) = w(P[s, y])$.

Correctitud de Dijkstra: por qué se fija el mínimo

Sea u el vértice de menor etiqueta en Q y sea P un camino mínimo de s a u. Como los pesos son no negativos, elegimos P simple.

1. Sea y el primer vértice de P que no está en S, y sea x su predecesor. Entonces $x \in S$.
2. Al fijar x, se relajó (x, y). Por el invariante, $d[y] \leq \delta(s, x) + w(x, y) = w(P[s, y])$.
3. Como el sufijo de P desde y hasta u tiene peso no negativo, $w(P[s, y]) \leq w(P) = \delta(s, u)$. Además, $d[u] \leq d[y]$ porque u tiene la menor etiqueta pendiente.

Correctitud de Dijkstra: por qué se fija el mínimo

Sea u el vértice de menor etiqueta en Q y sea P un camino mínimo de s a u. Como los pesos son no negativos, elegimos P simple.

1. Sea y el primer vértice de P que no está en S, y sea x su predecesor. Entonces $x \in S$.
2. Al fijar x, se relajó (x, y). Por el invariante, $d[y] \leq \delta(s, x) + w(x, y) = w(P[s, y])$.
3. Como el sufijo de P desde y hasta u tiene peso no negativo, $w(P[s, y]) \leq w(P) = \delta(s, u)$. Además, $d[u] \leq d[y]$ porque u tiene la menor etiqueta pendiente.

Correctitud de Dijkstra: por qué se fija el mínimo

Sea u el vértice de menor etiqueta en Q y sea P un camino mínimo de s a u. Como los pesos son no negativos, elegimos P simple.

1. Sea y el primer vértice de P que no está en S, y sea x su predecesor. Entonces $x \in S$.
2. Al fijar x, se relajó (x, y). Por el invariante, $d[y] \leq \delta(s, x) + w(x, y) = w(P[s, y])$.
3. Como el sufijo de P desde y hasta u tiene peso no negativo, $w(P[s, y]) \leq w(P) = \delta(s, u)$. Además, $d[u] \leq d[y]$ porque u tiene la menor etiqueta pendiente.

Por lo tanto,

$$\delta(s, u) \leq d[u] \leq d[y] \leq w(P[s, y]) \leq \delta(s, u),$$

y así $d[u] = \delta(s, u)$. El invariante se conserva al agregar u a S.

Complejidad y límite de Dijkstra

Con listas de adyacencia y una cola de prioridad implementada con un heap binario:

- Se ejecuta EXTRACTMIN una vez por vértice: $O(n \log n)$.
- Se examina cada arista una vez; las disminuciones de prioridad cuestan $O(\log n)$ cada una: $O(m \log n)$.

Complejidad y límite de Dijkstra

Con listas de adyacencia y una cola de prioridad implementada con un heap binario:

- Se ejecuta ExtractMin una vez por vértice: $O(n \log n)$.
- Se examina cada arista una vez; las disminuciones de prioridad cuestan $O(\log n)$ cada una: $O(m \log n)$.

Complejidad

Tiempo $O((n+m)\log n)$ y espacio $O(n+m)$. Con un arreglo sin heap, seleccionar el mínimo cuesta $O(n)$ por iteración y el total es $O(n^2+m)$, y si el grafo es simple el tiempo es $O(n^2)$.

Complejidad y límite de Dijkstra

Con listas de adyacencia y una cola de prioridad implementada con un heap binario:

- Se ejecuta ExtractMin una vez por vértice: $O(n \log n)$.
- Se examina cada arista una vez; las disminuciones de prioridad cuestan $O(\log n)$ cada una: $O(m \log n)$.

Complejidad

Tiempo $O((n+m)\log n)$ y espacio $O(n+m)$. Con un arreglo sin heap, seleccionar el mínimo cuesta $O(n)$ por iteración y el total es $O(n^2+m)$, y si el grafo es simple el tiempo es $O(n^2)$.

Limitación

Si hay aristas de peso negativo, la prueba de deja de valer. Para esos grafos usamos Bellman–Ford.

Aplicación: recorridos acotados

Aplicación

Sea $G = (V, E, w)$ un digrafo con pesos no negativos. Dados s, t y una cota C, buscamos una arista de peso máximo que aparezca en algún recorrido de s a t cuyo peso total sea a lo sumo C.

Aplicación: recorridos acotados

Aplicación

Sea $G = (V, E, w)$ un digrafo con pesos no negativos. Dados s, t y una cota C, buscamos una arista de peso máximo que aparezca en algún recorrido de s a t cuyo peso total sea a lo sumo C.

Idea: concatenar tres tramos

Para una arista $e = (u, v)$, el recorrido más barato que va de s a t y usa e cuesta $\delta(s, u) + w(e) + \delta(v, t)$. Por lo tanto, e puede aparecer en un recorrido permitido exactamente cuando esta suma es a lo sumo C. Entre esas aristas elegimos la de mayor peso.

Algoritmo

1. Ejecutar Dijkstra desde s en G y obtener $d_s[u] = \delta(s, u)$.
2. Invertir las aristas y ejecutar Dijkstra desde t; así obtenemos $d_t[v] = \delta(v, t)$ en el grafo original.
3. Para cada $e = (u, v) \in E$, si $d_s[u] + w(e) + d_t[v] \leq C$, considerar e y conservar la de mayor peso.
4. Si ninguna arista pasa el filtro, informar que no existe una.

Algoritmo

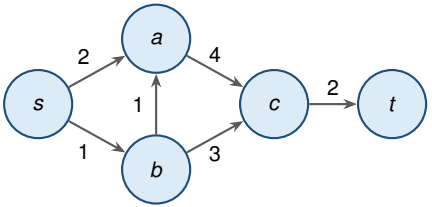
1. Ejecutar Dijkstra desde s en G y obtener $d_s[u] = \delta(s, u)$.
2. Invertir las aristas y ejecutar Dijkstra desde t; así obtenemos $d_t[v] = \delta(v, t)$ en el grafo original.
3. Para cada $e = (u, v) \in E$, si $d_s[u] + w(e) + d_t[v] \leq C$, considerar e y conservar la de mayor peso.
4. Si ninguna arista pasa el filtro, informar que no existe una.

Complejidad

Dos ejecuciones de Dijkstra y un barrido de las aristas: $O((n + m) \log n)$ con heap binario y listas de adyacencia.

Ejemplo: distancias desde s y hacia t

Tomamos la cota $C = 7$.



Distancias calculadas

v	$\delta(s, v)$	$\delta(v, t)$
s	0	6
a	2	6
b	1	5
c	4	2
t	6	0

Ejemplo: filtramos las aristas

Una arista $e = (u, v)$ se admite si $\delta(s, u) + w(e) + \delta(v, t) \leq 7$.

Arista e	Peso mínimo de un recorrido que la usa	¿Admisible?
$s \to a$	$0 + 2 + 6 = 8$	No
$s \to b$	$0 + 1 + 5 = 6$	Sí
$b \to a$	$1 + 1 + 6 = 8$	No
$a \to c$	$2 + 4 + 2 = 8$	No
$b \to c$	$1 + 3 + 2 = 6$	Sí
$c \to t$	$4 + 2 + 0 = 6$	Sí

Respuesta

La arista admisible de mayor peso es $b \to c$, de peso 3.

Correctitud de la solución

Para cada arista $e = (u, v)$:

- Si un recorrido permitido usa e, su prefijo hasta u cuesta al menos $\delta(s, u)$ y su sufijo desde v cuesta al menos $\delta(v, t)$. Entonces $\delta(s, u) + w(e) + \delta(v, t) \leq C$.
- Si la suma es a lo sumo C, concatenamos un camino mínimo de s a u, la arista e y un camino mínimo de v a t. Obtenemos un recorrido permitido que contiene e.

Así, el filtro conserva exactamente las aristas que pueden aparecer en un recorrido permitido. Elegir entre ellas la de mayor peso devuelve la respuesta óptima; si el conjunto es vacío, no existe una arista buscada.

Caminos mínimos entre todos los pares

Floyd–Warshall calcula las distancias mínimas entre todo par de vértices. Admite aristas negativas, siempre que el digrafo no tenga ciclos de peso negativo.

Inicialización

$D^{(0)}[i,j] = 0$ si $i = j$; el menor peso de una arista $i \to j$ si existe; y $+\infty$ si no hay una arista directa.

Para $k = 1, \ldots, n$, permitimos que v_k aparezca como vértice interno:

$$D^{(k)}[i,j] = \text{mín}\left(D^{(k-1)}[i,j],\ D^{(k-1)}[i,k] + D^{(k-1)}[k,j]\right).$$

En cada paso comparamos el mejor camino conocido con uno que pasa por v_k. La complejidad temporal es $O(n^3)$. Mejor que Bellman – Ford cuando $m = O(n^2)$.

Correctitud de Floyd–Warshall

Invariante

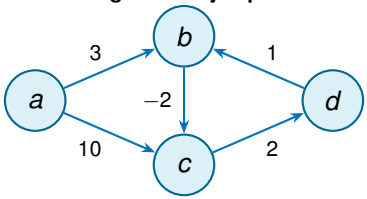
Luego de procesar $v_1, \ldots, v_k$, $D[i, j]$ es el peso mínimo de un camino de i a j cuyos vértices internos pertenecen a $\{v_1, \ldots, v_k\}$.

- **Base:** con $k = 0$, solo se permiten aristas directas (y el camino vacío de un vértice a sí mismo); eso es exactamente $D^{(0)}$.
- **Paso:** un camino óptimo no usa v_k como vértice interno, o lo usa y se divide en un tramo $i \rightsquigarrow v_k$ y otro $v_k \rightsquigarrow j$. La recurrencia toma el mínimo de ambos casos.

Como no hay ciclos negativos, un camino mínimo puede elegirse simple: quitar cualquier ciclo no aumenta su peso. Así, el argumento por inducción cubre un camino mínimo entre cada par alcanzable.

Animación: incorporamos vértices intermedios

Digrafo de ejemplo



Matriz de distancias

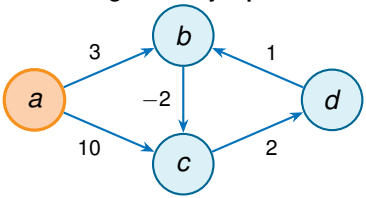
$D^{(0)}$ (aristas directas)

	a	b	c	d
a	0	3	10	∞
b	∞	0	-2	∞
c	∞	∞	0	2
d	∞	1	∞	0

Inicio: solo conocemos las aristas directas.

Animación: incorporamos vértices intermedios

Digrafo de ejemplo



Matriz de distancias

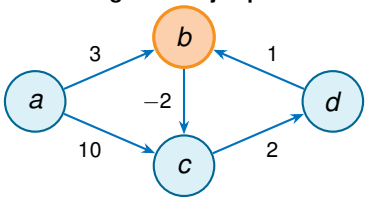
$D^{(1)}$ (intermedio a)

	a	b	c	d
a	0	3	10	∞
b	∞	0	-2	∞
c	∞	∞	0	2
d	∞	1	∞	0

No hay aristas que entren a a: no cambia la matriz.

Animación: incorporamos vértices intermedios

Digrafo de ejemplo



Matriz de distancias

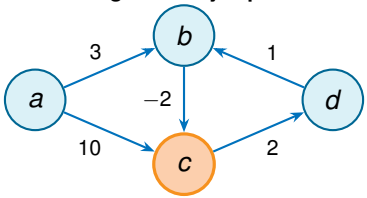
$D^{(2)}$ (intermedios a, b)

	a	b	c	d
a	0	3	**1**	∞
b	∞	0	-2	∞
c	∞	∞	0	2
d	∞	1	$-$**1**	0

$a \to b \to c$ y $d \to b \to c$ mejoran dos entradas.

Animación: incorporamos vértices intermedios

Digrafo de ejemplo



Matriz de distancias

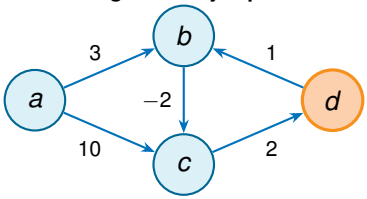
$D^{(3)}$ (intermedios a, b, c)

	a	b	c	d
a	0	3	1	**3**
b	∞	0	-2	**0**
c	∞	∞	0	2
d	∞	1	-1	0

Pasando por c, mejoran las distancias hacia d.

Animación: incorporamos vértices intermedios

Digrafo de ejemplo



Matriz de distancias

$D^{(4)}$ (todos los intermedios)

	a	b	c	d
a	0	3	1	3
b	∞	0	-2	0
c	∞	**3**	0	2
d	∞	1	-1	0

Pasando por d, $c \to d \to b$ mejora $D[c, b]$.

Aplicación: la arista más eficiente

Aplicación

Dado un digrafo sin ciclos de costo negativo o cero, buscamos la arista $e = (u, v)$ que sea *st-eficiente* para la mayor cantidad de pares ordenados (s, t).

Aplicación: la arista más eficiente

Aplicación

Dado un digrafo sin ciclos de costo negativo o cero, buscamos la arista $e = (u, v)$ que sea *st-eficiente* para la mayor cantidad de pares ordenados (s, t).

Una vez calculadas todas las distancias $D[x, y]$, contamos para cada arista:

$$\text{score}(u, v) = |\{(s, t) : D[s, u] + w(u, v) + D[v, t] = D[s, t]\}| .$$

Aplicación: la arista más eficiente

Aplicación

Dado un digrafo sin ciclos de costo negativo o cero, buscamos la arista $e = (u, v)$ que sea *st-eficiente* para la mayor cantidad de pares ordenados (s, t).

Una vez calculadas todas las distancias $D[x, y]$, contamos para cada arista:

$$\text{score}(u, v) = |\{(s, t) : D[s, u] + w(u, v) + D[v, t] = D[s, t]\}| .$$

Lectura de la igualdad

El mejor tramo de s a u, la arista $u \to v$ y el mejor tramo de v a t tienen juntos el mismo costo que un camino mínimo de s a t.

Solo contamos pares para los que las tres distancias involucradas son finitas.

Algoritmo

1. Ejecutar Floyd–Warshall y obtener $D[s, t]$ para todo par.
2. Para cada arista $e = (u, v)$, recorrer los pares (s, t) alcanzables y contar aquellos que satisfacen $D[s, u] + w(e) + D[v, t] = D[s, t]$.
3. Devolver la arista con mayor contador.

Algoritmo

1. Ejecutar Floyd–Warshall y obtener $D[s, t]$ para todo par.
2. Para cada arista $e = (u, v)$, recorrer los pares (s, t) alcanzables y contar aquellos que satisfacen $D[s, u] + w(e) + D[v, t] = D[s, t]$.
3. Devolver la arista con mayor contador.

Complejidad

$O(n^3 + mn^2)$ tiempo y $O(n^2)$ espacio. Si $m = O(n^2)$, entonces el tiempo total es $O(n^3)$.

Correctitud de la aplicación

Para una arista $e = (u, v)$ y un par (s, t) alcanzable:

- Si existe un recorrido mínimo de s a t que usa e, su prefijo hasta u y su sufijo desde v también son mínimos; si no, se podría reemplazar uno y obtener un recorrido de menor peso. Luego se cumple la igualdad.
- Si se cumple la igualdad, concatenar un camino mínimo de s a u, la arista e y un camino mínimo de v a t produce un recorrido mínimo de s a t que usa e.

Correctitud de la aplicación

Para una arista $e = (u, v)$ y un par (s, t) alcanzable:

- Si existe un recorrido mínimo de s a t que usa e, su prefijo hasta u y su sufijo desde v también son mínimos; si no, se podría reemplazar uno y obtener un recorrido de menor peso. Luego se cumple la igualdad.
- Si se cumple la igualdad, concatenar un camino mínimo de s a u, la arista e y un camino mínimo de v a t produce un recorrido mínimo de s a t que usa e.

Entonces el contador mide exactamente los pares para los que existe un recorrido mínimo usando cada arista. Devolver el contador máximo resuelve el problema.

Ciclos de peso cero

Cuando se permiten ciclos de peso cero puede ocurrir lo siguiente.

$$s \to u\ (0), \qquad e = u \to v\ (0), \qquad v \to u\ (0), \qquad u \to t\ (1)$$

Aquí $D[s,u] + w(e) + D[v,t] = 0 + 0 + 1 = D[s,t]$, pero el único camino simple mínimo de s a t es $s \to u \to t$ y no usa e.

Resumen

- Bellman–Ford admite pesos negativos y detecta ciclos negativos alcanzables.
- Dijkstra resuelve caminos mínimos cuando los pesos son no negativos.
- Floyd–Warshall calcula distancias entre todos los pares y permite aristas negativas si no hay ciclos negativos.
- Se puede usar distancias para filtrar aristas en recorridos.
- Si una regla depende del historial, incorporamos esa información al estado.